

AWX Custom UI Deployment Guide

Server: 192.168.10.32

Date: October 14, 2025

AWX Version: 24.6.1 (Custom)

AWX Operator: 2.19.1

Table of Contents

- [Prerequisites](#)
 - [Phase 1: Clean Up Previous Installation](#)
 - [Phase 2: Setup Kind Cluster](#)
 - [Phase 3: Deploy AWX Operator](#)
 - [Phase 4: Build Custom AWX Image](#)
 - [Phase 5: Deploy Custom AWX](#)
 - [Phase 6: Configure CORS and LDAP](#)
 - [Troubleshooting](#)
-

Prerequisites

Server Requirements:

- OS: Linux (CentOS/RHEL/Rocky)
- Docker: v28.4.0+
- Kind: installed
- kubectl: configured
- Git: installed
- Minimum 8GB RAM, 50GB disk space

Local Machine (Windows):

- Node.js: v22.12.0+
- Git: with SSH key configured
- AWX source code: <https://github.com/thehien0121/awx.git> (branch: 24.6.1-docs)

Network:

- Server IP: 192.168.10.32
 - NodePort: 32000
 - SSH Access: root@192.168.10.32
-

Phase 1: Clean Up Previous Installation

1.1 Delete Existing AWX Instance

```
cd /home/docker/awx/awx-operator
kubectl delete -f awx-cr.yaml
```

1.2 Delete AWX Operator

```
kubectl delete -k .
```

1.3 Delete Kind Cluster

```
kind delete cluster
kind get clusters # Verify deletion
```

1.4 Clean Up Docker Images (Optional)

```
# List old images
docker images | grep awx

# Remove old custom images
docker rmi awx-custom-ui:20251010
docker rmi awx-assistant:20240721

# Clean dangling images
docker image prune -f
```

⚠ Note: Keep [quay.io/ansible/awx-ee:latest](https://quay.io/repository/ansible/awx-ee/latest) - it's required for job execution!

Phase 2: Setup Kind Cluster

2.1 Create Kind Configuration

```
cd /home/docker/awx

cat > kind.config << 'EOF'
apiVersion: kind.x-k8s.io/v1alpha4
kind: Cluster
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 32000
    hostPort: 32000
    listenAddress: "0.0.0.0"
    protocol: tcp
```

```
- role: worker
EOF
```

2.2 Create Kind Cluster

```
kind create cluster --config=kind.config
```

Expected Output:

```
Creating cluster "kind" ...
✓ Ensuring node image (kindest/node:v1.31.0)
✓ Preparing nodes 📦 📦
✓ Writing configuration 📄
✓ Starting control-plane 🎮
✓ Installing CNI 🔧
✓ Installing StorageClass 💾
✓ Joining worker nodes 🚚
Set kubectl context to "kind-kind"
```

2.3 Verify Cluster

```
kubectl cluster-info --context kind-kind
```

2.4 Install NGINX Ingress Controller

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-
nginx/main/deploy/static/provider/kind/deploy.yaml

# Wait for ingress controller to be ready
kubectl wait --namespace ingress-nginx \
  --for=condition=ready pod \
  --selector=app.kubernetes.io/component=controller \
  --timeout=90s
```

Phase 3: Deploy AWX Operator

3.1 Create Namespace

```
kubectl create namespace awx
kubectl config set-context --current --namespace=awx
```

3.2 Clone AWX Operator

```
cd /home/docker/awx
git clone https://github.com/ansible/awx-operator.git
cd awx-operator
git checkout 2.19.1
```

3.3 Create Kustomization File

```
cat > kustomization.yaml << 'EOF'
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - github.com/ansible/awx-operator/config/default?ref=2.19.1

images:
  - name: quay.io/ansible/awx-operator
    newTag: 2.19.1

namespace: awx
EOF
```

3.4 Deploy Operator

```
kubectl apply -k .

# Wait for operator to be ready
kubectl wait --for=condition=available --timeout=300s deployment/awx-operator-
controller-manager -n awx
```

Verify:

```
kubectl get pods -n awx
```

Expected: `awx-operator-controller-manager` pod running (2/2)

Phase 4: Build Custom AWX Image

4.1 Clone Source Code on Server

On Server (192.168.10.32):

```
cd /home/docker/awx
mkdir -p awx-source-custom
cd awx-source-custom

# Clone your custom AWX repository
git clone https://github.com/thehien0121/awx.git .

# Checkout the custom branch
git checkout 24.6.1-docs

# Verify source code
git log --oneline -3
```

4.2 Generate Dockerfile

```
cd /home/docker/awx/awx-source-custom

# Generate Dockerfile and supervisor configs from template
make Dockerfile
* After make Dockerfile, change openssl-3.0.7(to old) to openssl
```

Expected Output:

```
PLAY RECAP *****
localhost : ok=4 changed=3 unreachable=0 failed=0
```

Verify:

```
ls -la _build/
# Should see: supervisor_web.conf, supervisor_task.conf, supervisor_rsyslog.conf
```

4.3 Build Custom AWX Docker Image

⚠ Important: Use **HEADLESS=no** to ensure collectstatic runs!

```
docker build -f Dockerfile \
  --build-arg VERSION=24.6.1 \
  --build-arg SETUPTOOLS_SCM_PRETEND_VERSION=24.6.1 \
  --build-arg HEADLESS=no \
  -t awx-custom:24.6.1 .
```

Build Time: ~3-5 minutes (with cache)

Expected Output:

```
[+] Building 101.6s (42/42) FINISHED
=> exporting to image
=> => naming to docker.io/library/awx-custom:24.6.1
```

4.4 Verify Image Contents

```
# Check UI files in image
docker run --rm awx-custom:24.6.1 \
  ls -la /var/lib/awx/venv/awx/lib/python3.11/site-
packages/awx/ui/build/static/js/ \
  | grep main

# Should see: main.*.js file (~3.3MB)
```

4.5 Load Image into Kind Cluster

```
kind load docker-image awx-custom:24.6.1 --name kind
```

Verify:

```
docker exec -it kind-control-plane crictl images | grep awx-custom
# Should see: docker.io/library/awx-custom 24.6.1
```

Phase 5: Deploy Custom AWX

5.1 Create Basic AWX CR (Without ConfigMaps)

```
cd /home/docker/awx/awx-operator

cat > awx-cr-basic.yaml << 'EOF'
apiVersion: awx.ansible.com/v1beta1
kind: AWX
metadata:
  name: awx-demo
spec:
  service_type: nodeport
  nodeport_port: 32000
  image: awx-custom
  image_version: "24.6.1"
  image_pull_policy: Never
```

```
postgres_image: postgres
postgres_image_version: "14"
auto_upgrade: false
EOF

kubectl create -f awx-cr-basic.yaml
```

Run db migration

```
kubectl exec -it deployment/awx-demo-web -n awx -c awx-demo-web -- \
  awx-manage migrate
```

5.2 Monitor Deployment

```
# Watch pods
kubectl get pods -n awx -w
```

Expected Pods:

- `awx-demo-postgres-15-0` (1/1 Running)
- `awx-demo-migration-*` (Completed)
- `awx-demo-web-*` (3/3 Running)
- `awx-demo-task-*` (4/4 Running)
- `awx-operator-controller-manager-*` (2/2 Running)

5.3 Verify Service

```
kubectl get svc -n awx
```

Expected:

```
awx-demo-service    NodePort    10.96.x.x    <none>    80:32000/TCP
```

5.4 Get Admin Password

```
kubectl get secret awx-demo-admin-password -o jsonpath="{.data.password}" -n awx |
base64 --decode ; echo
```

Default username: `admin`

5.5 Test Access

Open browser: <http://192.168.10.32:32000/>

Should see AWX login page.

Phase 6: Configure CORS and LDAP

6.1 Delete Current AWX Instance

```
kubectl delete awx awx-demo -n awx
```

6.2 Create CORS ConfigMap

```
cd /home/docker/awx/awx-operator
```

```
cat > dev_cors.py << 'EOF'
```

```
CORS_ALLOWED_ORIGINS = [  
    "http://localhost:3001",  
    "http://127.0.0.1:3001",  
    "http://192.168.10.46:3001",  
    "http://222.253.44.37:3001",  
]
```

```
CORS_ALLOW_CREDENTIALS = True
```

```
CORS_ALLOW_HEADERS = [  
    'accept',  
    'accept-encoding',  
    'authorization',  
    'content-type',  
    'dnt',  
    'origin',  
    'user-agent',  
    'x-csrftoken',  
    'x-requested-with',  
]
```

```
CORS_ALLOW_METHODS = [  
    'DELETE',  
    'GET',  
    'OPTIONS',  
    'PATCH',  
    'POST',  
    'PUT',  
]
```

```
CSRF_TRUSTED_ORIGINS = [  
    "http://localhost:3001",
```

```

    "http://127.0.0.1:3001",
    "http://192.168.10.46:3001",
    "http://192.168.10.46:32000",
    "http://222.253.44.37:3001",
]

CSRF_COOKIE_SECURE = False
CSRF_COOKIE_SAMESITE = 'Lax'

SESSION_COOKIE_SECURE = False
SESSION_COOKIE_SAMESITE = 'Lax'

CORS_ORIGIN_ALLOW_ALL = False
EOF

kubectl create configmap dev-cors-settings --from-file=dev_cors.py=dev_cors.py -n
awx

```

6.3 Create LDAP ConfigMap

```

cat > custom_ldap_settings.py << 'EOF'
import ldap
from django_auth_ldap.config import LDAPSearch, PosixGroupType

AUTH_LDAP_SERVER_URI = "ldap://vn3.g-days.net:389"

AUTH_LDAP_BIND_DN = "cn=ldapadm,dc=mydomain,dc=com"
AUTH_LDAP_BIND_PASSWORD = "kjkj1234"

AUTH_LDAP_USER_SEARCH = LDAPSearch(
    "ou=People,dc=mydomain,dc=com",
    ldap.SCOPE_SUBTREE,
    "(uid=%(user)s)"
)

AUTH_LDAP_GROUP_SEARCH = LDAPSearch(
    "ou=Group,dc=mydomain,dc=com",
    ldap.SCOPE_SUBTREE,
    "(objectClass=posixGroup)"
)

AUTH_LDAP_GROUP_TYPE = PosixGroupType(name_attr="memberUid")
AUTH_LDAP_REQUIRE_GROUP = "cn=AWX,ou=Group,dc=mydomain,dc=com"

AUTH_LDAP_USER_FLAGS_BY_GROUP = {
    "is_active": "cn=AWX,ou=Group,dc=mydomain,dc=com",
    "is_superuser": "cn=AWX_admin,ou=Group,dc=mydomain,dc=com",
}

AUTH_LDAP_USER_DN_TEMPLATE = None
AUTH_LDAP_BIND_AS_AUTHENTICATING_USER = False

```

```

AUTHENTICATION_BACKENDS = (
    'django_auth_ldap.backend.LDAPBackend',
    'django.contrib.auth.backends.ModelBackend',
)
EOF

kubectl create configmap custom-ldap-settings --from-
file=custom_ldap_settings.py=custom_ldap_settings.py -n awx

```

6.4 Create Final AWX CR with ConfigMaps

```

cat > awx-cr-final.yaml << 'EOF'
apiVersion: awx.ansible.com/v1beta1
kind: AWX
metadata:
  name: awx-demo
spec:
  service_type: nodeport
  nodeport_port: 32000
  image: awx-custom
  image_version: "24.6.1"
  image_pull_policy: Never
  postgres_image: postgres
  postgres_image_version: "14"
  auto_upgrade: false
  extra_volumes: |
    - name: dev-cors-settings
      configMap:
        name: dev-cors-settings
        items:
          - key: dev_cors.py
            path: dev_cors.py
    - name: custom-ldap-settings
      configMap:
        name: custom-ldap-settings
        items:
          - key: custom_ldap_settings.py
            path: custom_ldap_settings.py
  web_extra_volume_mounts: |
    - name: dev-cors-settings
      mountPath: /etc/tower/conf.d/dev_cors.py
      subPath: dev_cors.py
      readOnly: true
    - name: custom-ldap-settings
      mountPath: /etc/tower/conf.d/custom_ldap_settings.py
      subPath: custom_ldap_settings.py
      readOnly: true
  task_extra_volume_mounts: |
    - name: dev-cors-settings
      mountPath: /etc/tower/conf.d/dev_cors.py

```

```
    subPath: dev_cors.py
    readOnly: true
  - name: custom-ldap-settings
    mountPath: /etc/tower/conf.d/custom_ldap_settings.py
    subPath: custom_ldap_settings.py
    readOnly: true
```

EOF

6.5 Deploy AWX with Full Configuration

```
kubectl create -f awx-cr-final.yaml
```

6.6 Monitor Deployment

```
# Watch deployment progress
kubectl get pods -n awx -w

# View operator logs
kubectl logs -f deployment/awx-operator-controller-manager -c awx-manager -n awx
```

Wait for all pods to be Running:

- awx-demo-postgres-15-0: 1/1 Running
- awx-demo-web-*: 3/3 Running
- awx-demo-task-*: 4/4 Running
- awx-demo-migration-*: Completed

Verification & Access

Check Service

```
kubectl get svc awx-demo-service -n awx
```

Access AWX UI

URL: <http://192.168.10.32:32000/>

Default Login:

- Username: `admin`
- Password: Get from secret (see Phase 5.4)

Verify Custom Features

1. **CORS Configuration:** Check browser console for CORS headers
 2. **LDAP Authentication:** Try login with LDAP user from `ou=People,dc=mydomain,dc=com`
 3. **Custom UI Features:**
 - Chat streaming
 - Auto restart socket
 - Job output bug fixes
-

Troubleshooting

Issue 1: Pods CrashLoopBackOff

Check logs:

```
kubectl logs <pod-name> -n awx -c <container-name>
```

Common causes:

- ConfigMap not found
- Incorrect image_pull_policy
- Database connection issues

Issue 2: UI Not Loading (404 errors)

Run collectstatic manually:

```
kubectl exec -it deployment/awx-demo-web -n awx -c awx-demo-web -- \
  awx-manage collectstatic --noinput --clear

# Restart web pod
kubectl rollout restart deployment/awx-demo-web -n awx
```

Issue 3: LDAP Not Working

Check LDAP connectivity from pod:

```
kubectl exec -it deployment/awx-demo-web -n awx -c awx-demo-web -- \
  ldapsearch -x -H ldap://vn3.g-days.net:389 -D "cn=ldapadm,dc=mydomain,dc=com" -w
  kjkj1234 -b "ou=People,dc=mydomain,dc=com"
```

Issue 4: Node.js Version Conflicts

During build on server:

- ✗ Don't use `make ui-devel` (requires Node 18+)
- ☒ Use `HEADLESS=no` in Docker build (installs correct Node.js version inside container)

Issue 5: Permission Denied on collectstatic

If collectstatic fails with permission errors:

```
kubectl exec -it deployment/awx-demo-web -n awx -c awx-demo-web -- \
  rm -rf /var/lib/awx/public/static/*

kubectl exec -it deployment/awx-demo-web -n awx -c awx-demo-web -- \
  awx-manage collectstatic --noinput
```

Important Notes

Docker Build Arguments

HEADLESS Mode:

- **HEADLESS=yes**: Skip UI build + skip collectstatic (faster but no static files)
- **HEADLESS=no**: Build UI + run collectstatic (required for production)

For Custom UI Deployment: Always use **HEADLESS=no** to ensure:

1. Pre-built UI is processed
2. collectstatic copies files to /var/lib/awx/public/static/
3. Nginx can serve static files correctly

Image Naming Convention

Use semantic versioning for images:

```
awx-custom:24.6.1      # Production
awx-custom:24.6.1-v2   # Testing iteration 2
awx-custom:24.6.1-dev  # Development
```

Git Workflow

Repository: <https://github.com/thehien0121/awx.git>

Branches:

- **devel**: Upstream sync branch
- **24.6.1-docs**: Custom features branch (ACTIVE)

Custom Commits:

- **bc9166ad80**: Add pre-built UI for fast deployment
- **0aec722063**: deployment 10.10.2025
- **100d2d4871**: fix bug showing job output

- `6dc44c3d6e`: chat streaming
- `0ecae1c23e`: add auto restart socket

Resource Requirements

Minimum:

- CPU: 4 cores
- RAM: 8GB
- Disk: 50GB

Recommended:

- CPU: 8 cores
- RAM: 16GB
- Disk: 100GB

Quick Reference Commands

Check Status

```
# All pods
kubectl get pods -n awx

# Services
kubectl get svc -n awx

# AWX CR
kubectl get awx -n awx

# Logs
kubectl logs -f deployment/awx-demo-web -n awx -c awx-demo-web
```

Restart AWX

```
# Restart web
kubectl rollout restart deployment/awx-demo-web -n awx

# Restart task
kubectl rollout restart deployment/awx-demo-task -n awx

# Full restart
kubectl delete awx awx-demo -n awx
kubectl create -f awx-cr-final.yaml
```

Update ConfigMaps

```
# Delete old ConfigMap
kubectl delete configmap dev-cors-settings -n awx

# Create new ConfigMap
kubectl create configmap dev-cors-settings --from-file=dev_cors.py=dev_cors.py -n
awx

# Restart to pick up changes
kubectl rollout restart deployment/awx-demo-web -n awx
kubectl rollout restart deployment/awx-demo-task -n awx
```

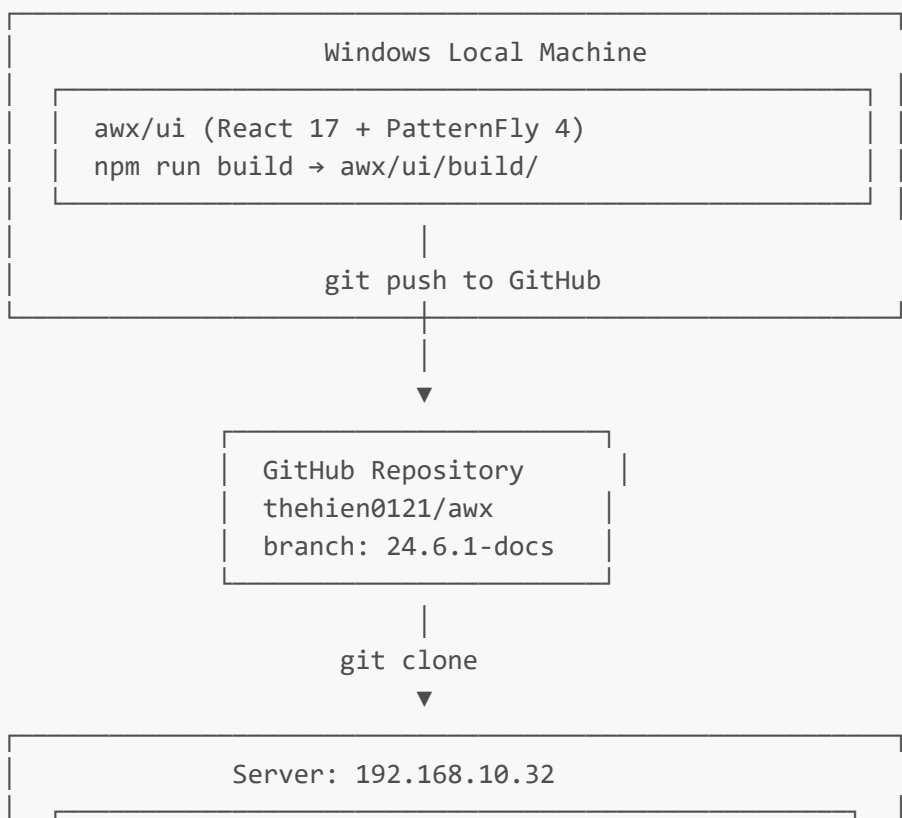
Clean Up Everything

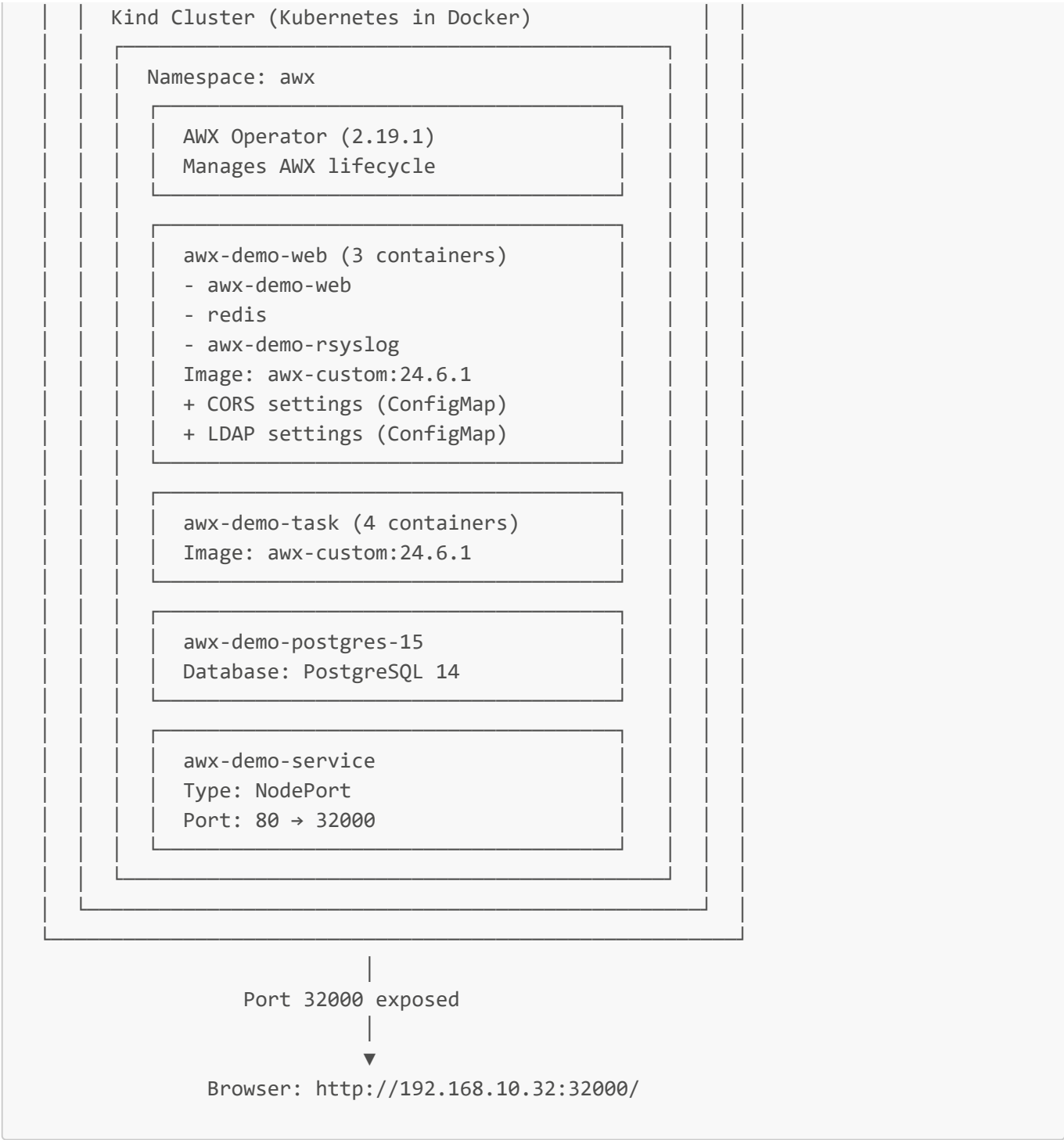
```
# Delete AWX
kubectl delete awx awx-demo -n awx

# Delete Operator
cd /home/docker/awx/awx-operator
kubectl delete -k .

# Delete Kind cluster
kind delete cluster
```

Architecture Diagram





Build Process Flow



Step 2: Generate Dockerfile & Build Configs

```
make Dockerfile
→ Dockerfile (generated from .j2 template)
→ _build/supervisor_*.conf
```



Step 3: Docker Build (Multi-stage)

Stage 1 (builder):

- Install Node.js 20.18.1 (via npm + n)
- COPY . /tmp/src/ (all source code)
- npm install && npm run build (build UI)
→ awx/ui/build/ created inside container
- make sdist && pip install
- collectstatic (copy UI to public/static/)

Stage 2 (final):

- COPY --from=builder /var/lib/awx
→ Includes built UI + collected static files
- Add supervisor configs
- Configure runtime
→ awx-custom:24.6.1 (860MB)



Step 4: Load into Kind & Deploy

```
kind load docker-image awx-custom:24.6.1
kubectl create -f awx-cr-final.yaml
→ AWX running with custom UI
```

Key Learnings

1. HEADLESS Mode Behavior

- **HEADLESS=yes:** Skips both npm build AND collectstatic
- **HEADLESS=no:** Runs npm build AND collectstatic
- **For custom UI:** Always use **HEADLESS=no**

2. UI Build Process

- **UI is built entirely inside Docker container** - no need to build on Windows/local machine
- Node.js 20.18.1 is automatically installed inside container via **npm install -g n && n 20.18.1**
- Build process:

1. `COPY . /tmp/src/` - copies all source code (including UI source)
 2. `npm install --legacy-peer-deps` - installs UI dependencies
 3. `npm run build` - builds production UI bundle
 4. `collectstatic` - collects all static files to `/var/lib/awx/public/static/`
- Build time: ~40-45 seconds for UI compilation
 - Total build time: ~5-10 minutes (with cache)
 - **Why not pre-build?** Docker build always runs `npm run build` when `HEADLESS=no`, which would overwrite any pre-built UI

3. Dockerfile is Auto-generated

```
### This file is generated from
### tools/ansible/roles/dockerfile/templates/Dockerfile.j2
###
### DO NOT EDIT
###
```

- Never edit Dockerfile directly
- Edit template at: `tools/ansible/roles/dockerfile/templates/Dockerfile.j2`
- Regenerate with: `make Dockerfile`

4. ConfigMap Best Practices

- Store configuration files as ConfigMaps
- Mount to `/etc/tower/conf.d/` for AWX to auto-load
- Changes require pod restart to take effect

5. Kind Cluster Considerations

- Use `image_pull_policy: Never` for local images
- Port mappings must be configured in `kind.config` before cluster creation
- Images must be loaded with `kind load docker-image`

Maintenance

Update Custom UI

1. Make UI changes locally:

```
cd /home/docker/awx/awx-source-custom/awx/ui/src
# Edit React components, styles, etc.
```

2. Commit changes:

```
cd /home/docker/awx/awx-source-custom
git add awx/ui/src/
git commit -m "Update UI: <description>"
git push origin 24.6.1-docs
```

3. Rebuild Docker image:

```
cd /home/docker/awx/awx-source-custom
git pull origin 24.6.1-docs

# Regenerate Dockerfile if template changed
make Dockerfile

# Build with new UI changes
docker build -f Dockerfile \
  --build-arg VERSION=24.6.1 \
  --build-arg SETUPTOOLS_SCM_PRETEND_VERSION=24.6.1 \
  --build-arg HEADLESS=no \
  -t awx-custom:24.6.1-new .

kind load docker-image awx-custom:24.6.1-new --name kind
```

4. Update AWX deployment:

```
cd /home/docker/awx/awx-operator
kubectl delete awx awx-demo -n awx

# Edit awx-cr-final.yaml to use new image tag
sed 's/24.6.1/24.6.1-new/' awx-cr-final.yaml > awx-cr-updated.yaml
kubectl create -f awx-cr-updated.yaml
```

Update ConfigMaps

```
# Edit configuration file
vi dev_cors.py

# Delete old ConfigMap
kubectl delete configmap dev-cors-settings -n awx

# Create new ConfigMap
kubectl create configmap dev-cors-settings --from-file=dev_cors.py=dev_cors.py -n awx

# Restart pods to load new config
kubectl rollout restart deployment/awx-demo-web -n awx
kubectl rollout restart deployment/awx-demo-task -n awx
```

Backup & Restore

Backup AWX Data:

```
# Create AWXBackup CR
cat > awx-backup.yaml << 'EOF'
apiVersion: awx.ansible.com/v1beta1
kind: AWXBackup
metadata:
  name: awx-backup-$(date +%Y%m%d)
  namespace: awx
spec:
  deployment_name: awx-demo
EOF

kubectl create -f awx-backup.yaml
```

Restore:

```
# Check backup
kubectl get awxbackup -n awx

# Create AWXRestore CR
cat > awx-restore.yaml << 'EOF'
apiVersion: awx.ansible.com/v1beta1
kind: AWXRestore
metadata:
  name: awx-restore
  namespace: awx
spec:
  deployment_name: awx-demo
  backup_name: awx-backup-20251014
EOF

kubectl create -f awx-restore.yaml
```

Performance Optimization

Scale Web and Task Pods

```
spec:
  web_replicas: 2
  task_replicas: 2
```

Enable HPA (Horizontal Pod Autoscaler)

```
spec:
  web_resource_requirements:
    requests:
      cpu: 500m
      memory: 1Gi
    limits:
      cpu: 1000m
      memory: 2Gi
```

Persistent Storage for Projects

```
spec:
  projects_persistence: true
  projects_storage_class: standard
  projects_storage_size: 20Gi
```

Security Considerations

Production Checklist

- ☐ Change default admin password
- ☐ Enable HTTPS/TLS
- ☐ Set `CSRF_COOKIE_SECURE = True`
- ☐ Set `SESSION_COOKIE_SECURE = True`
- ☐ Set `CORS_ORIGIN_ALLOW_ALL = False`
- ☐ Use strong LDAP bind password
- ☐ Enable Network Policies
- ☐ Configure Resource Limits
- ☐ Enable Pod Security Policies
- ☐ Regular backups scheduled

For Production Deployment

1. Use proper image registry:

```
spec:
  image: registry.example.com/awx-custom
  image_version: "24.6.1"
  image_pull_policy: Always
  image_pull_secrets:
    - registry-credentials
```

2. **Enable TLS:**

```
spec:
  ingress_type: ingress
  ingress_tls_secret: awx-tls-cert
  hostname: awx.example.com
```

3. **Use external PostgreSQL:**

```
spec:
  postgres_configuration_secret: external-postgres-config
```

Contact & Support

Repository: <https://github.com/thehien0121/awx>
Branch: 24.6.1-docs
Server: 192.168.10.32
Documentation: <https://ansible.readthedocs.io/projects/awx-operator/>

Changelog

Date	Version	Changes
2025-10-14	24.6.1	☒ Successfully deployed AWX custom on Kind cluster
		☒ Integrated CORS and LDAP ConfigMaps
		☒ Resolved HEADLESS mode and collectstatic issues
		☒ Verified UI build process inside Docker container
2025-10-10	-	Added deployment script improvements
Earlier	-	Fixed job output display bug
		Implemented chat streaming feature
		Added auto restart socket functionality

Summary

What We Achieved:

- ☒ Clean installation of Kind cluster with AWX Operator 2.19.1
- ☒ Custom AWX 24.6.1 image built with UI customizations
- ☒ CORS configuration for cross-origin requests
- ☒ LDAP authentication integration

- ☒ Fully functional AWX accessible at <http://192.168.10.32:32000/>

Key Success Factors:

1. Using `HEADLESS=no` to ensure UI build and collectstatic run correctly
2. Following AWX's official build workflow (`make Dockerfile` → `docker build`)
3. Proper ConfigMap integration for runtime configuration
4. Understanding Docker multi-stage build process

Total Setup Time: ~30-40 minutes (including build)

End of Setup Guide